

The Mediator-Role Test as Standalone Operational Procedure: Verifying AI-as-Substrate-Mediator Compliance in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **mediator-role test** — the standalone operational procedure that verifies whether a CKS deployment satisfies the AI-as-substrate-mediator commitment of §4.2 — as a composite of five Property-specific checks corresponding to the five operational properties of the mediator role.

Abstract

The CKS pattern names AI-as-substrate-mediator as a core-theory commitment whose operational content is a five-property role specification: the LLM reads substrate content as primary state, writes only under human-authored orchestration rules, holds no substrate-relevant state outside the substrate, exercises no authority over substrate content, and has its writes recorded in the substrate with attribution. Whether a deployment satisfies the commitment is not self-evident from any single observation. This note formalizes the **mediator-role test** as a composite operational procedure with five Property checks (A through E), one per property; states what the test outputs (pass / fail with Property-specific failure identification); enumerates which anti-patterns each Property check detects; specifies how the test integrates into deployment verification; and states its architectural limits. The note is the fifth Phase A5 operational test note and opens the cluster covering AI-mediation and substrate-state architecture.

1. Why the mediator-role test needs to be formalized as standalone

The AI-as-substrate-mediator commitment is the cross-claim spine of the CKS theory: §4.2 of the source paper identifies it as the commitment that ties the substrate/LLM governance boundary, the human-governed authority architecture, conflict preservation, tool-agnosticism, linear-cost scaling, and the multi-human extension into one design posture. The commitment is composite — five operational properties together specify what the LLM is and is not authorized to do — and any one of those properties can fail independently. A deployment can satisfy four properties, violate one, and still

misrepresent itself as CKS-coherent if no test names the failure precisely.

The standalone test addresses three problems. First, the five-property structure is not exercisable as a single observation: a deployment might appear to read from substrate content (Property A) while still holding substrate-relevant state externally (Property C). Each Property must be checked independently. Second, the anti-patterns the source paper rules out cluster around specific Properties, and a unified test that does not decompose by Property cannot identify which anti-pattern the deployment exhibits. Third, deployment events such as activation, composition, LLM vendor change, and cell-architecture revision each require re-verification of the role, and a procedural form is what makes the re-verification routine rather than ad hoc.

This is the fifth Phase A5 operational test note. It opens the cluster covering AI-mediation and substrate-state architecture, following the four-governance-rights cluster.

2. The architectural commitment under test

The test verifies the AI-as-substrate-mediator commitment of §4.2 — the role specifying that the LLM mediates over a human-governed substrate rather than holding authority, producing terminal artifacts, or maintaining authoritative state outside the substrate. The commitment decomposes into five operational properties (Properties A through E) that hold for every operation the LLM performs, at all times during the deployment's existence:

- **Property A — substrate-content reads as primary state.** The LLM reads from substrate content as its primary source of state for the operation it performs. - **Property B — substrate-content writes under orchestration rules.** The LLM writes to substrate content only under orchestration rules authored by humans. - **Property C — no substrate-relevant state outside the substrate.** The LLM does not hold substrate-relevant state outside the substrate — neither in agent memory persisted across invocations, nor in per-session storage treated as authoritative, nor in in-weight memory of substrate content treated as the answer to "what is the case." - **Property D — no authority over substrate content.** The LLM does not exercise authority over substrate content: no silent overwrite, no collapse of preserved contradictions, no modification of orchestration rules. - **Property E — LLM writes recorded with attribution.** Substrate writes resulting from an LLM operation are recorded as substrate content with attribution sufficient to identify them as LLM-authored and to associate them with the orchestration rule under which they were produced.

Property A follows from §4.1's governance boundary; Property B from §4.1 in conjunction with the human-governed commitment; Property C from §11.3's substrate-as-source-of-truth treatment; Property D from the human-governed commitment as it applies to non-human operators; Property E from the path-retraceability commitment of §3.1.

3. The test procedure: five Property checks

The test is composite. It consists of five Property checks, each independently exercisable, that together verify the mediator role. Each check is stated as an operational question whose answer is determined by inspection of the deployment's architecture and behavior, not by interview, documentation review, or

vendor attestation.

Property A check — substrate reads. For each cell that uses an LLM, the check verifies that substrate-relevant inputs to the LLM call originate at substrate reads performed by the cell. The cell may pass non-substrate inputs alongside (the user's request, the orchestration rule, transient working state), but substrate-relevant state must come from the substrate. A cell whose inputs are constructed from agent memory, from a per-session cache of prior LLM invocations, or from in-weight model knowledge treated as substitutive for substrate state fails the check. Exercise: trace data flow into the LLM call and verify substrate-relevant inputs originate at substrate reads.

Property B check — rule-mediated writes. For each cell, the check verifies that LLM outputs reach the substrate only through a transformation step governed by an orchestration rule authored by humans. The transformation may be a simple pass-through, but the rule must exist as a substrate-level artifact and the path from LLM output to substrate write must be traceable through it. A cell whose LLM output is written directly to substrate state, or whose rule is itself LLM-authored without prior human authority, fails the check.

Property C check — no external substrate state. For each cell, the check verifies that no substrate-relevant state persists across cell executions outside the substrate. Transient state held during a single execution is admissible; state surviving across executions and substrate-relevant must live in the substrate. Exercise: enumerate the deployment's persistent storage layers and verify that no substrate-relevant state lives in agent memory, per-session storage held across invocations, vendor-side conversation history treated as authoritative, in-weight model memory used substitutively for substrate, or per-cell hidden state not also substrate-readable.

Property D check — no authority exercise. For each cell, the check verifies that the LLM does not perform actions reserved for humans by the human-governed commitment — silent overwrite of substrate content, collapse of preserved contradictions outside what orchestration rules authorize, or modification of orchestration rules. Authority-exercising operations are exercisable only by humans through the modify and override rights, or by orchestration rules humans have authored. Exercise: enumerate the operations the LLM is permitted to invoke and verify that none carries authority semantics over substrate content or rules.

Property E check — attribution. For each substrate write, the check verifies that the write is recorded in the substrate's provenance with attribution sufficient to identify whether it was LLM-produced, human-produced, or rule-produced; and, for LLM-produced writes, with attribution sufficient to identify the orchestration rule and the cell execution within which the operation occurred. Exercise: inspect the substrate's provenance and verify that LLM contributions are distinguishable from human and rule-engine contributions.

Each check is exercised at the cell level (Properties A–D) or at the substrate level (Property E), independently of the others.

4. What the test outputs

The test outputs a binary pass/fail result per Property check and an aggregate pass/fail result for the deployment.

Each Property check outputs a binary result — Property satisfied or Property failed — together with, in the failure case, an enumeration of the cells (Properties A–D) or substrate-write classes (Property E) that fail the check. The failure enumeration is what makes remediation possible: the operator knows which cells violate which Property and can restrict remediation to those cells.

The deployment satisfies the test only if all five Property checks pass. A failure on any single Property is a failure of the test in aggregate, regardless of how many other Properties are satisfied. The aggregate output is binary; it does not blend per-Property results into a score, because the role is composite — a deployment satisfying four Properties and failing one is not a four-fifths-mediated deployment, it is a deployment that exhibits the anti-pattern associated with the failed Property.

5. What anti-patterns the test detects

The five Property checks specifically detect six anti-patterns the source paper's mediator-role specification rules out, with each Property check covering a distinct cluster.

Property A failure detects LLM-as-autonomous-agent (substrate-bypass aspect). When the LLM reads substrate-relevant state from agent memory, prior conversation history, or in-weight model knowledge rather than from the substrate, the deployment is treating the LLM as an autonomous agent maintaining its own model of the world.

Property B failure detects LLM-as-terminal-producer (canonical detection). When LLM outputs flow to substrate state without rule mediation, the LLM has become the terminal producer of substrate content rather than a mediator over it.

Property C failure detects three state-related anti-patterns: agent memory as source of truth, LLM context as source of truth, and hidden cell state. Each places substrate-relevant state outside the substrate. Property C is the catchment for all three; the failure enumeration distinguishes them by the location of the offending state.

Property D failure detects LLM-as-source-of-truth (canonical detection) and LLM-as-autonomous-agent (authority aspect). When the LLM exercises authority over substrate content — silently overwriting, collapsing contradictions, modifying rules — the deployment is treating the LLM as the holder of authority the human-governed commitment reserves for humans.

Property E failure detects audit-trail violations specific to LLM contributions. When LLM contributions are not distinguishable from human contributions in provenance — attribution missing, generic, or unable to tie a write back to its originating cell execution and rule — the audit trail necessary for path retraceability is broken at the LLM-contribution slice.

The coverage is by design coextensive with the five Properties: no anti-pattern the source paper rules out under the mediator-role specification escapes detection.

6. How the test integrates with deployment verification

The test is exercisable at five lifecycle events, each of which can change whether the role still holds.

Initial deployment validation. Before activation, the test is run as part of pre-activation verification. A deployment that fails any Property check is not activated until remediation closes the failure. The initial run establishes a baseline subsequent runs check against.

Composition-partner verification. When a deployment composes with another substrate, the test is re-run on the composed deployment. A composition in which each component independently passes its checks can still produce a composed deployment in which the role is violated by emergent paths that did not exist in either component alone.

Periodic audit. Organizational policy may require periodic re-verification on the schedule the deployment's compliance regime imposes. The periodic run is the same test, exercised on the deployment's then-current state.

LLM-vendor or model-update verification. The role is a property of the deployment's architecture, not of the specific LLM the deployment uses; the tool-agnosticism commitment of §7.1 provides for substitution without architectural change. But the substitution is itself an event after which the role must be re-verified: a new LLM may carry capabilities (memory, tool-use, agent-style behavior) that previous architectural assumptions did not anticipate.

Cell-architecture-change verification. When cell logic is modified — a cell added, an orchestration rule rewritten, input or output paths restructured — the test is re-run on the affected cells. The check is exercisable per-cell, so re-verification need not re-test the deployment as a whole if the change is scoped.

The five integration points make the role exercisable as a routine deployment property rather than as a one-time architectural claim.

7. Limits of the test

The mediator-role test verifies the mediator role specifically. Five categories of property are out of scope.

Path retraceability beyond Property E. The substrate-wide path retraceability of §3.1 is the subject of a separate Phase A5 operational test (provenance completeness). Property E covers the LLM-attribution slice; it does not cover human-write attribution, rule-execution attribution, or cross-cell propagation attribution.

Determinism. The cell-behavior-determinism property and the read-determinism property are subjects of separate Phase A5 operational tests. Mediator-role compliance is necessary but not sufficient for those guarantees, which depend on additional properties of orchestration rules and substrate read semantics.

Substrate-as-source-of-truth beyond Property D. The five-categories source-of-truth specification of §11.3 is the subject of a separate Phase A5 operational test. Property D covers authority over substrate content as it applies to LLM operations; it does not cover the substrate's status as authoritative across all five categories the source paper enumerates.

Composition properties. The composition framework's five requirements are the subject of a separate Phase A5 operational test. The mediator-role test is exercised on a single deployment (or on a composed deployment as a single object); it does not exercise the composition contract that holds between two component deployments.

LLM output correctness or rule logic correctness. The test verifies architectural placement — that the LLM is doing the work the role specifies, in the location it specifies, with the inputs and outputs it specifies. It does not verify that the LLM produces good outputs, that orchestration rules express good policy, or that the cell's overall behavior is fit for purpose. A deployment that passes the test can still produce poor outputs from a poorly written orchestration rule operating over an LLM that does the wrong thing within the right architectural envelope.

The narrowness is deliberate. A test that conflates architectural placement with semantic correctness loses its diagnostic value.

8. One-sentence test

A CKS deployment satisfies the mediator-role test if and only if, for every cell that uses an LLM, the cell reads substrate-relevant inputs from substrate content (Property A), routes LLM outputs to substrate writes only through human-authored orchestration rules (Property B), holds no substrate-relevant state across executions outside the substrate (Property C), grants the LLM no authority to overwrite, collapse, or modify rules (Property D), and records LLM-produced substrate writes in provenance with attribution sufficient to identify the LLM's contribution and the rule under which it operated (Property E).

9. Why naming this test as standalone matters

The mediator role is the most consequential single architectural commitment in the CKS theory: §4.2 names it the cross-claim spine. A commitment that load-bearing requires a verification procedure exercisable independently of the other CKS verification activities. Formalizing the test as a standalone composite procedure makes three things possible. Deployments can be verified for mediator-role compliance specifically, without having to pass the entire battery of CKS operational tests at once. Mediator-role failures can be identified specifically, with the failed Property pointing to a specific cluster of anti-patterns and a specific remediation site. The test's limits are explicit, so an operator who runs the test knows what it does and does not certify; mediator-role compliance is necessary but not sufficient for the full set of CKS commitments, and conflating the two would let mediator-compliant-but-otherwise-non-compliant deployments pass under the wrong banner.

This is the fifth Phase A5 operational test note and the first in the cluster covering AI-mediation and substrate-state architecture. Subsequent Phase A5 notes cover cell-behavior-determinism,

read-determinism, provenance-completeness, four-accountability-questions, source-of-truth-five-categories, tool-agnosticism-migration, linear-cost-scaling, conflict-coexistence, composition-requirements-five, pattern-mapping, and reproducibility tests. Together with the four-governance-rights cluster, these tests constitute the operational verification framework that makes CKS commitments exercisable in deployment rather than only defensible in argument.

Subsequent work that adopts the CKS pattern, audits a deployment for compliance, or argues that a system is CKS-coherent should exercise the mediator-role test as formalized here, with results expressed in the Property-specific vocabulary the test produces. Work that asserts mediator-role compliance without exercising the five Property checks — or that conflates mediator-role compliance with the broader CKS verification battery — is using a different verification standard, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Mediator-Role Test as Standalone Operational Procedure: Verifying AI-as-Substrate-Mediator Compliance in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.